

Desafio de Auditoria: Sistema de Gestão Supermercado

PARTE 2 — Implementação e Expansão

Após a correção coletiva dos erros identificados na auditoria (Parte 1), as quatro missões abaixo expandem o sistema com boas práticas de POO: encapsulamento com validação, herança, polimorfismo e atualização do diagrama de classes.

Missão 1 — Blindagem de Dados

Getters e setters foram implementados para todos os atributos de **Produto**. Os métodos **setPreco()** e **setEstoque()** rejeitam valores negativos, exibem mensagem de erro e mantêm o valor original.

```
// Classe Produto - atributos protegidos (corrigido: private ->
protected) class Produto {    protected String nome;
    protected double preco;    // corrigido: float -> double
    protected int estoque;

    public Produto(String n, double p, int e) { // construtor
completo        this.nome = n;        this.preco = p;
this.estoque = e;    }

    // --- Getters ---
    public String getNome()    { return nome; }
    public double getPreco()    { return preco; }
    public int    getEstoque(){ return estoque; }

    // --- Setters com validacao --    public void setPreco(double novoPreco) {
if (novoPreco < 0) {                System.out.println("Erro: preco nao pode ser
negativo. Valor mantido.");        } else {
        this.preco =
novoPreco;        }    }

    public void setEstoque(int novoEstoque) {        if (novoEstoque < 0) {
System.out.println("Erro: estoque nao pode ser negativo. Valor mantido.");
    } else {
        this.estoque =
novoEstoque;        }    }

    public void adicionarEstoque(int qtd) { estoque += qtd; }

    public void vender(int qtd) {        if (estoque >= qtd) {
estoque -= qtd;        System.out.println("Vendido: " + qtd + "
unidade(s) de " + nome);
        } else {        System.out.println("Estoque
insuficiente para " + nome);
        }
    }
}
```

Conceito aplicado: Encapsulamento — o acesso direto a atributos é bloqueado; a validação de regras de negócio ocorre dentro do setter.

Missão 2 — Especialização (ProdutoImportado)

A classe **ProdutoImportado** herda de **Produto**, adiciona o atributo **taxaImportacao** (float) e sobrescreve o método **vender()** para exibir o preço final já com a taxa aplicada.

```
class ProdutoImportado extends Produto {
    private float taxaImportacao;

    public ProdutoImportado(String n, double p, int e, float taxa) {
        super(n, p, e); // chama construtor da superclasse
        this.taxaImportacao = taxa;
    }

    public float getTaxaImportacao() { return taxaImportacao; }

    @Override
    public void vender(int qtd) {
        double precoFinal = preco * (1 + taxaImportacao / 100.0);
        System.out.println("Produto importado: " + nome);          System.out.printf("Preço com
        taxa (%.1f%%): R$ %.2f%n", taxaImportacao, precoFinal);      if (estoque >= qtd) {
        estoque -= qtd;          System.out.printf("Vendido: %d unidade(s). Total: R$ %.2f%n",
        qtd, precoFinal * qtd);      } else {          System.out.println("Estoque
        insuficiente para " + nome);
        }
    }
}
```

Conceito aplicado: Herança e Polimorfismo — ProdutoImportado reusa a estrutura de Produto e sobrescreve (@Override) o comportamento de vender() com lógica específica.

Missão 3 — O Gestor de Vendas (Classe Venda)

A classe **Venda** possui o método **processarItem(Produto p, int quantidade)** que aceita qualquer tipo de produto graças ao polimorfismo. Ele calcula o total (quantidade × preço) e chama o **vender()** correto de cada objeto dinamicamente. Uma **sobrecarga** do método (sem o parâmetro quantidade) também é fornecida.

```
class Venda {

    // Sobrescrita (polimorfismo): aceita Produto, Perecivel ou
    ProdutoImportado public void processarItem(Produto p, int quantidade) {
        System.out.println("--- Processando venda ---");          double total =
        p.getPreco() * quantidade;          System.out.printf("Produto: %s | Qtd: %d |
        Total estimado: R$ %.2f%n",
        p.getNome(), quantidade, total);
        p.vender(quantidade); // polimorfismo: chama vender() do tipo real do objeto
        System.out.println("-----");
    }

    // Sobrecarga: mesmo nome, parametro diferente (assume quantidade =
    1) public void processarItem(Produto p) {          processarItem(p,
    1);
    }
}
```

Diferença entre Sobrecarga e Sobrescrita:

Conceito	Definição	Exemplo no sistema
----------	-----------	--------------------

Sobrecarga (Overload)	Mesmo nome de método, parâmetros diferentes, mesma classe.	processarItem(Produto p) processarItem(Produto p, int qtd)
Sobrescrita (Override)	Mesmo nome e parâmetros, comportamento diferente na subclasse.	Produto.vender(int qtd) Perecivel.vender(int qtd) ProdutoImportado.vender(int qtd)

Classe de Execução — Supermercado.java

```
public class Supermercado {
    public static void main(String[] args) {

        // Produto normal - setPreco rejeita valor negativo
        Produto p1 = new Produto("Arroz", 25.50, 100);
        p1.setPreco(-10.0); // exibe erro, mantém R$ 25,50

        // Perecivel - construtor correto via super()
        Perecivel leite = new Perecivel("Leite", 5.0, 10, "10/05/2026");

        // ProdutoImportado - taxa de 30%
        ProdutoImportado vinho = new
        ProdutoImportado("Vinho Importado", 120.0, 5, 30f);

        Venda venda = new Venda();
        venda.processarItem(p1, 2); // vender() de Produto
        venda.processarItem(leite, 3); // vender() de Perecivel (polimorfismo)
        venda.processarItem(vinho, 1); // vender() de ProdutoImportado (polimorfismo)
        venda.processarItem(p1); // sobrecarga: qtd = 1
    }
}
```

Saida esperada no console:

```
Erro: preco nao pode ser negativo. Valor mantido.
--- Processando venda --Produto: Arroz | Qtd: 2
| Total estimado: R$ 51,00
Vendido: 2 unidade(s) de Arroz
-----
--- Processando venda --Produto: Leite | Qtd: 3
| Total estimado: R$ 15,00
Verificando validade: 10/05/2026 Item
perecivel vendido: 3 unidade(s) de Leite
-----
--- Processando venda --Produto: Vinho Importado | Qtd: 1 |
Total estimado: R$ 120,00
Produto importado: Vinho Importado
Preco com taxa (30,0%): R$ 156,00
Vendido: 1 unidade(s). Total: R$ 156,00
-----
```

Missão 4 — Diagrama de Classes Atualizado

